

## Описание функций сглаживания — Гибридная версия

---

Каждый метод описан математической формулой, за которой следует программная реализация на Python и C++.

### 1. Boxcar (Равномерное скользящее среднее)

#### 1.1 Математическая формула

Равномерное скользящее среднее по окну  $M$  (половина окна  $m = M // 2$ ):

$$\begin{aligned} N(i) &= \{ j \mid \max(0, i + 1 - m) \leq j < \min(m + i, N) \} \\ W_i &= |N(i)| \\ y[i] &= (1 / W_i) \cdot \sum_{j \in N(i)} x[j] \end{aligned}$$

На границах массива окно обрезается (clamping).

#### 1.2 Реализация — Python

Boxcar.py:2–12

```
def boxcar(win, data):
    half = win // 2
    n = len(data)
    avg = []
    for i in range(n):
        start = max(0, i + 1 - half)
        end = min(half + i, n)
        sum_array = sum(data[start:end])
        aver = sum_array / len(data[start:end])
        avg.append(aver)
    return avg
```

#### 1.3 Реализация — C++

calculator.h:31–50

```
// Дополнение отражением + префиксные суммы
inline void movingAverage(const vector<double>& in,
                          vector<double>& out, int win) {
    int n = (int)in.size();
    int half = win / 2;
    out.resize(n);

    vector<double> padded(n + 2 * half);
    for (int i = 0; i < half; i++) {
        padded[i] = in[half - 1 - i];
        padded[n + half + i] = in[n - 1 - i];
    }
    for (int i = 0; i < n; i++)
        padded[half + i] = in[i];

    vector<double> prefix(padded.size() + 1, 0.0);
    for (int i = 0; i < (int)padded.size(); i++)
        prefix[i + 1] = prefix[i] + padded[i];

    for (int i = 0; i < n; i++)
        out[i] = (prefix[i + win] - prefix[i]) / win;
}
```

## 1.4 Свойства

| Аспект            | Значение                                       |
|-------------------|------------------------------------------------|
| Веса              | Равные: $1/W_i$ для всех элементов в окне      |
| АЧХ               | $H(f) = \sin(\pi f M) / (M \cdot \sin(\pi f))$ |
| Граничные условия | Python: clamping   C++: mirror reflection      |
| Суммирование      | Python: прямое   C++: префиксные суммы         |

## 2. Gaussian (Гауссов фильтр)

### 2.1 Математическая формула

Ядро Гаусса на окне  $M$  (половина окна  $m = M // 2$ ,  $\sigma = M / 6$ ):

$$\begin{aligned}
 w[k] &= \exp\left(-\frac{(k - m)^2}{2\sigma^2}\right), \quad k = 0, 1, \dots, M - 1 \\
 w^-[k] &= w[k] / \sum_{j=0}^{M-1} w[j] \\
 y[i] &= \sum_{k=0}^{M-1} w^-[k] \cdot x^-[i + k]
 \end{aligned}$$

Где  $x^-$  — сигнал, дополненный симметрическим отражением на  $m$  элементов с каждой стороны.

### 2.2 Реализация ядра — Python

Gaussian.py:4–11

```
def gaussian(data, win):
    half = win // 2
    sigma = win / 6.0
    n = len(data)
    k = [math.exp(-0.5 * (i - half)**2 / sigma**2)
          for i in range(win)]
    k = [x / sum(k) for x in k]
    p = ([data[i] for i in range(half-1, -1, -1)]
          + data
          + [data[i] for i in range(n-1, n-half-1, -1)])
    return [sum(k[j] * p[i+j] for j in range(win))
            for i in range(n)]
```

## 2.3 Реализация ядра — C++

calculator.h:54–67

```
vector<double> gaussianKernel(int win) {
    int half = win / 2;
    double sigma = win / 6.0;
    vector<double> kernel(win);
    double sum = 0;
    for (int i = 0; i < win; i++) {
        int d = i - half;
        kernel[i] = exp(-0.5 * (d * d) / (sigma * sigma));
        sum += kernel[i];
    }
    for (int i = 0; i < win; i++)
        kernel[i] /= sum;
    return kernel;
}
```

## 2.4 Реализация свёртки — C++

calculator.h:70–93

```
inline void gaussianFilter(const vector<double>& in,
                          vector<double>& out,
                          const vector<double>& kernel) {
    int n = (int)in.size();
    int win = (int)kernel.size();
    int half = win / 2;
    out.resize(n);

    vector<double> padded(n + 2 * half);
    for (int i = 0; i < half; i++) {
        padded[i] = in[half - 1 - i];
        padded[n + half + i] = in[n - 1 - i];
    }
    for (int i = 0; i < n; i++)
        padded[half + i] = in[i];

    for (int i = 0; i < n; i++) {
        double sum = 0;
        for (int j = 0; j < win; j++)
            sum += kernel[j] * padded[i + j];
        out[i] = sum;
    }
}
```

## 2.5 Свойства

| Аспект            | Значение                                     |
|-------------------|----------------------------------------------|
| Параметр sigma    | $M / 6$ — идентичен в Python и C++           |
| Граничные условия | Mirror reflection — идентичны                |
| Структура кода    | Python: единая функция   C++: ядро + свёр... |
| Сложность         | $O(N \cdot M)$                               |

## 3. Progressive (Прогрессивное среднее)

### 3.1 Математическая формула

$$y[i] = (1 / (i + 1)) \cdot \sum_{k=0}^i x[k], \quad i = 0, 1, \dots, N - 1$$

Рекуррентно:  $S[i] = S[i - 1] + x[i], \quad y[i] = S[i] / (i + 1)$

### 3.2 Реализация — Python

Progressive.py:3–9

```
def progressive(data):
    result = []
    total = 0.0
    for i, value in enumerate(data):
        total += value
        result.append(total / (i + 1))
    return result
```

### 3.3 Реализация — C++

calculator.h:97–105

```
inline void progressiveMean(const vector<double>& in,
                           vector<double>& out) {
    int n = (int)in.size();
    out.resize(n);
    double sum = 0;
    for (int i = 0; i < n; i++) {
        sum += in[i];
        out[i] = sum / (i + 1);
    }
}
```

### 3.4 Свойства

| Аспект                | Значение                                |
|-----------------------|-----------------------------------------|
| Размер окна           | $i + 1$ (растёт от 1 до $N$ )           |
| Дисперсия             | $O(1/i)$ — минимум на последнем отсчёте |
| Совпадение реализаций | Полное — идентичная логика              |

## 4. Hybrid (Гибридное сглаживание)

### 4.1 Математическая формула

```

y[i] =
    y boxcar [i],                если  $0 \leq i < M$ 
     $(1 - \lambda) \cdot y \text{ boxcar } [i] + \lambda \cdot y \text{ prog } [i]$ ,    если  $M \leq i < 2M$ 
    y prog [i],                  если  $i \geq 2M$ 
 $\lambda(i) = (i - M) / M, \quad \lambda \in [0, 1)$ 

```

## 4.2 Реализация — Python

Hybrid (Boxcar + Progressive).py:21–37

```

def hybrid(data, win):
    n = len(data)
    box = boxcar(win, data)
    prog = progressive(data)

    out = []
    for i in range(n):
        if i < win:
            out.append(box[i])
        elif i < 2 * win:
            z = (i - win) / win
            k = (1.0 - z) * box[i] + z * prog[i]
            out.append(k)
        else:
            out.append(prog[i])
    return out

```

## 4.3 Реализация — C++

calculator.h:110–135

```

inline void hybridSmooth(const vector<double>& in,
                        vector<double>& out, int win) {
    int n = (int)in.size();
    out.resize(n);

    vector<double> boxcar;
    movingAverage(in, boxcar, win);

    vector<double> progressive;
    progressiveMean(in, progressive);

    for (int i = 0; i < n; i++) {
        if (i < win) {
            out[i] = boxcar[i];
        } else if (i < 2 * win) {
            double w = (double)(i - win) / win;
            out[i] = (1.0 - w) * boxcar[i] + w * progressive[i];
        } else {
            out[i] = progressive[i];
        }
    }
}

```

## 4.4 Зоны и свойства

| Зона | Диапазон        | Поведение                                     |
|------|-----------------|-----------------------------------------------|
| I    | $0 \leq i < M$  | Boxcar: стабильное усреднение при малом чи... |
| II   | $M \leq i < 2M$ | Линейный crossfade: непрерывный переход       |
| III  | $i \geq 2M$     | Progressive: минимальная дисперсия при нак... |

| Аспект                   | Значение                                              |
|--------------------------|-------------------------------------------------------|
| Коэффициент интерполяции | $\lambda(i) = (i - M) / M$ — идентичен в Python и C++ |
| Пороговые значения       | $M$ и $2M$ — идентичны                                |
| Непрерывность            | Гарантирована на всех границах зон                    |